Beyond Development: Explaining Climate Resilience Across Indian Districts with a Boosted Attentive Climate Push Network

Sehjodh Grewal
Department of Networking and Communications, School of Computing
SRM Institute of Science & Technology
Kattankulathur, Chennai, India
sg2827@srmist.edu.in

Devanshu Kumar
Department of Networking and Communications, School of Computing
SRM Institute of Science & Technology
Kattankulathur, Chennai, India
dk5504@srmist.edu.in

Deepankar
Department of Networking and Communications, School of Computing
SRM Institute of Science & Technology
Kattankulathur, Chennai, India
dz9629@srmist.edu.in

Tanmay Singh
Department of Networking and Communications, School of Computing
SRM Institute of Science & Technology
Kattankulathur, Chennai, India
ts4494@srmist.edu.in

Dr. Banu Priya Prathaban
Department of Networking and Communications, School of Computing
SRM Institute of Science & Technology
Kattankulathur, Chennai, India
banuprip2@srmist.edu.in
Corresponding Author

***Abstract*—Development indicators are often used as a shorthand for how well an Indian district can cope with climate stress, yet many developed districts remain highly vulnerable while some poorer ones are resilient. We study this gap for 469 districts in 21 states by combining the 2021 district climate vulnerability index, Census 2011 development variables and 30 years (1991–2020) of daily India Meteorological Department (IMD) rainfall and temperature grids, about 212 million gridded values. A single-node Hadoop pipeline stores the data in HDFS (1.08 GB), turns 7,013,120 district-day rows into 19,200 district-year climate indices with MapReduce on YARN, builds the 30-year baseline and the analysis table with Hive, and groups districts into climate regimes with Spark. On this table we propose B-ACPN, a Boosted Attentive Climate Push Network: a random-forest stage supplies a first risk score, and a small neural network adds a development term and a development-gated, attention-weighted *climate push*, averaged over five networks, with Monte Carlo Dropout and conformal uncertainty. On 25 repeated cross-validation folds B-ACPN reaches a ROC-AUC of 0.820, accuracy 0.738 and F1 0.728, the highest of six models; it is significantly better than random forest, a standard MLP, a linear SVM and logistic regression, and comparable to gradient-boosted trees ($p = 0.55$). Its climate push separates the anomalous districts from the expected ones ($p < 0.001$), showing that climate, not only development, explains why developed districts can stay vulnerable. Tuning over 18 settings changes ROC-AUC by less than 0.004, and the CEEW 2021 district ranking barely agrees with our target index (Spearman $\rho = 0.02$).**

***Index Terms*—climate vulnerability, India, Hadoop, MapReduce, Hive, Spark, deep learning, explainable machine learning, conformal prediction**

I. Introduction

India is among the countries most exposed to heavy rainfall, droughts and heat, and district administrations are the level at which adaptation is planned. National assessments rank districts by climate vulnerability [1], [2], and a common assumption is that more developed districts, with higher literacy and less dependence on farming, are less vulnerable. The data only partly agree. In our sample development explains just 13% of the variation in the vulnerability index (R^2 = 0.127), and 181 of 469 districts (38.6%) break the expected pattern: they are developed yet vulnerable, or less developed yet resilient.

Explaining these anomalies needs long, fine-grained climate records. Thirty years of daily IMD grids [4], [5] for the whole country are too large to handle comfortably as plain files, and the same steps (aggregating days into years, years into a baseline, and joining with development data) recur whenever the period or the variables change. We therefore built the processing as a Hadoop pipeline. On the modelling side, an ordinary classifier says whether a district is highly vulnerable but not *why*. We wanted a model that is accurate and that also separates the climate contribution from the development contribution for every district.

The main contributions of this work are:

- A reproducible Big Data pipeline in which HDFS, MapReduce on YARN, Hive and Spark each do a measured job, with every step checked against an independent implementation.
- A preprocessing workflow without imputation, evaluated with eight error metrics, including the error that a naive nearest-cell aggregation would have introduced.
- B-ACPN, a hybrid model that boosts a random-forest score with a development-gated attentive climate network and gives a per-district climate push with uncertainty.
- A fair comparison with five existing models on identical

folds, with 17 metrics, confidence intervals, corrected significance tests, an ablation study and a before/after analysis of preprocessing.

The rest of the paper is organised as follows. Section II reviews related work. Section III describes the data, the preprocessing and the Big Data pipeline. Section IV presents B-ACPN. Section V reports the experiments, and Section VI discusses limitations before Section VII concludes.

II. Related Work

A. Climate vulnerability in India

Mohanty and Wadhawan [1] mapped district-level exposure and vulnerability to hydro-meteorological extremes across India, and the Department of Science and Technology [2] published a common framework for district vulnerability assessment. The IPCC [3] stresses that vulnerability depends on both exposure and adaptive capacity. These assessments build composite indices; they do not ask why districts with similar development end up with different vulnerability, which is our focus.

B. Big Data platforms

HDFS [6] stores large files as replicated blocks, MapReduce [7] expresses batch aggregations as map and reduce functions, and YARN [8] schedules them. Hive [9] adds SQL over HDFS, and Spark [10] keeps data in memory, which suits iterative algorithms; its MLlib library [11] provides standard classifiers. We use each tool for the step it fits best rather than forcing all steps into one engine.

C. Learning on tabular data

For small tabular datasets, tree ensembles such as random forests [12] and gradient boosting [13] remain strong, and Grinsztajn et al. [14] showed that they still often beat deep networks. Attention over input features, as in TabNet [15], makes a network's use of features visible. Stacking [16] feeds one model's out-of-fold predictions to another. B-ACPN combines these ideas: the trees provide accuracy, and the network adds a structured, explainable correction.

D. Uncertainty

Monte Carlo Dropout [18] approximates Bayesian uncertainty by keeping dropout active at prediction time, and split conformal prediction [19] turns any classifier's scores into prediction sets with a coverage guarantee. Both are cheap and fit our small model.

III. Data and Big Data Pipeline

A. Sources

Table I lists the sources. The outcome is the 2021 district climate vulnerability index from NITI Aayog's National Data and Analytics Platform (NDAP, dataset 7160), split at its median into High and Low vulnerability. Five development variables come from Census 2011 (literacy, female literacy, agricultural-worker share) and NDAP (MGNREGA days, yield variability). Eight climate hazard variables are computed from the IMD grids. The index is only the target and is never an input.

TABLE I
Data sources

Dataset	Source	Year	Size	Role
Vulnerability index	NDAP 7160	2021	699×79	target, 2 dev.
Census abstract	Census	2011	640 dist.	3 dev.
Daily rainfall, 0.25°	IMD	1991–2020	763 MB	hazards
Daily Tmax/Tmin, 1°	IMD	1991–2020	84 MB	hazards
Boundaries	DataMeet	2011	641 polygons	weights

B. Preprocessing

Census counts were converted to rates and checked against national totals: the population matches the official figure exactly and national literacy differs by 0.0019 percentage points. NDAP and Census districts were matched inside state clusters (Andhra Pradesh with Telangana, Jammu and Kashmir with Ladakh) by exact name, an alias table and fuzzy matching; 602 of 699 NDAP districts were matched with no duplicate codes, and two wrong fuzzy matches were found by hand and blocked. NDAP is sparse (75% of all cells empty), so we kept only the five development variables that are well covered and dropped districts with missing values instead of imputing them, leaving 469 complete districts.

Grid values were turned into district values with area weights. The weight of grid cell c for district d is

$$w_{c,d} = \frac{A(c \cap d)\cos\varphi_c}{\sum_{c'} A(c' \cap d)\cos\varphi_{c'}}, \tag{1}$$

where $w_{c,d}$ is the area weight of grid cell c for district d, $A(c\cap d)$ is the share of cell c lying inside district d (computed in an equal-area projection), φ_c is the latitude of the centre of cell c (cells shrink towards the north), and the sum runs over every grid cell c' that overlaps district d, so the weights add up to one.

The weights of every district sum to one within 10^{-7}. Daily district values are weighted means over the cells with data that day; IMD missing codes stay missing and are never set to zero. An automatic check confirms the grid orientation: Chennai's rainfall must peak in the north-east monsoon (October–December). The result is a table of 7,013,120 district-day rows.

Table II evaluates the two preprocessing steps that have an exact reference with eight error metrics. Replacing our area-weighted rainfall by the naive nearest-cell value would cause an RMSE of 287 mm and a MAPE of 11.1% per district; the MapReduce indices agree with an independent pandas recomputation to rounding error.

C. Big Data pipeline

Fig. 1 shows the pipeline. It runs on a single-node, pseudo-distributed Hadoop 3.3.6 cluster (HDFS replication 1, 128 MB blocks, 2 vCPU, 12.7 GB RAM) in Google Colab, with Java 8, Hive 3.1.3 and Spark 3.5.3. Colab has no SSH, so each daemon is started directly. Because the cluster has one node we make no claim about multi-machine speed-up.

TABLE II
PREPROCESSING ERROR METRICS

Metric	Nearest cell vs ours (mm)	MapReduce vs pandas*
RMSE	286.9	0.003
MSE	8.2×10^4	8.4×10^{-6}
MAE	151.7	0.003
MAPE	11.08%	0.008%
R^2	0.904	0.999999
NRMSE	0.034	1.9×10^{-4}
MedAE	87.0	0.002
Bias (ME)	5.045	-2.2×10^{-5}

Left: annual rainfall of 636 districts, 2020; reference = our area-weighted value.
*Worst of the seven indices over 19,200 district-years.

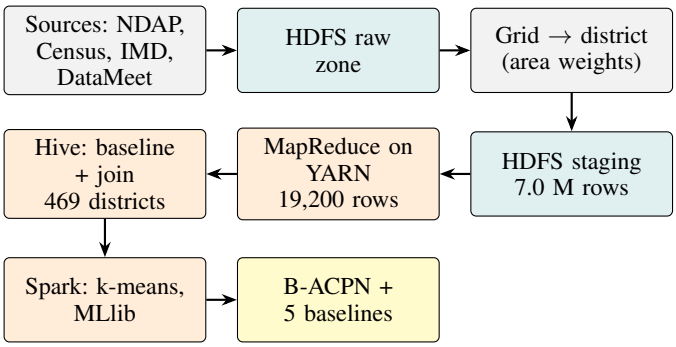


Fig. 1. Processing pipeline. Teal: HDFS storage zones; orange: Big Data processing; yellow: models.

HDFS holds three zones: *raw* for the untouched sources, *staging* for the district-day table and *processed* for job outputs. MapReduce, run with Hadoop Streaming, keys every district-day row by (district, year); each reducer sorts the days of one district-year by date and computes seven indices: annual rainfall, heavy-rain days (rain $\geq$ 64.5 mm), maximum five-day rainfall, longest dry spell (rain $<$ 1 mm), mean daily maximum temperature, hot days ($T_{\max} \geq 35\,^{\circ}\mathrm{C}$) and tropical nights ($T_{\min} \geq 25\,^{\circ}\mathrm{C}$). Incomplete years are left blank rather than set to zero. Hive then averages each index over 1991–2020, adds the inter-annual standard deviation of rainfall as rain variability, and joins the result with the development table on the Census district code. Spark k-means on the eight standardised hazard variables, with k chosen by the silhouette score [21], groups districts into climate regimes. Table III summarises the jobs.

TABLE III
BIG DATA JOBS (SINGLE NODE, COLAB RUN)

Job	Input	Output	Result
HDFS load	raw + staging	3 zones	1.08 GB
MapReduce	7,013,120 rows	19,200 rows	333 s
Hive	19,200 rows + dev.	469 districts	= pandas
Spark k-means	8 hazards	2 regimes	silh. 0.651

IV. PROPOSED MODEL: B-ACPN

A. Overview

Let $\mathbf{x}_i = (\mathbf{d}_i, \mathbf{h}_i)$ be the five development and eight hazard variables of district i and $y_i \in \{0, 1\}$ its vulnerability class. B-ACPN (Fig. 2) has three parts.

Tree stage. A random forest [12] with 500 trees and a minimum leaf size of two is trained on all 13 features. For training districts its probability is computed out-of-fold over five inner folds, so the next stage never sees a score that the forest produced on its own training rows. The tree score of district i is the log-odds of its random-forest probability,

$$s_i = \log \frac{\hat{p}_i^{\mathrm{RF}}}{1 - \hat{p}_i^{\mathrm{RF}}}, \tag{2}$$

where s_i is the tree score of district i and $\hat{p}_i^{\mathrm{RF}}$ is the random-forest probability that district i is highly vulnerable, computed out-of-fold for training districts.

Deep stage. A small network starts from s_i and adds a development term and a climate push. The baseline b_i, the attention weights $\mathbf{a}_i$ and the climate push c_i of district i are

$$b_i = s_i + \mathbf{w}^{\top}\mathbf{d}_i + \beta, \tag{3}$$

$$\mathbf{a}_i = \mathrm{softmax}\big(W_2 \tanh(W_1 \mathbf{h}_i)\big), \quad \tilde{\mathbf{h}}_i = 8\,\mathbf{a}_i \odot \mathbf{h}_i, \tag{4}$$

$$c_i = f_\theta(\tilde{\mathbf{h}}_i) \cdot 2\,\sigma\big(g_\phi(\mathbf{d}_i)\big). \tag{5}$$

where $\mathbf{d}_i$ and $\mathbf{h}_i$ are the five standardised development features and the eight standardised climate-hazard features of district i, $\mathbf{w}$ and β are the learned weights and bias of the development term, W_1 and W_2 are the learned weight matrices of the attention network, the attention weights $\mathbf{a}_i$ sum to one, $\tilde{\mathbf{h}}_i$ are the attention-weighted hazards (the factor 8 keeps the mean weight at one), $\odot$ is the element-wise product, f_θ is the climate network (an MLP with a residual block) with parameters θ, g_ϕ is the development-gate network with parameters ϕ, and $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid function; $c_i > 0$ means that climate adds risk and $c_i < 0$ that it protects.

Here $\mathbf{a}_i$ are attention weights over the hazards, f_θ is a linear layer (8→16) with ReLU and dropout 0.3, a residual block [17] (linear, batch normalisation, ReLU, dropout, skip connection) and a bias-free output layer, and the development gate $2\sigma(g_\phi)$, with g_ϕ a 5→4→1 tanh network, scales the push between 0 and 2. A positive push means that climate adds risk beyond what the trees and development explain.

Ensemble. Five deep stages with different seeds are averaged, so the final probability that district i is highly vulnerable is

$$\hat{p}_i = \frac{1}{M} \sum_{m=1}^{M} \sigma\big(b_i^{(m)} + c_i^{(m)}\big), \qquad M = 5. \tag{6}$$

where $\hat{p}_i$ is that final probability, M is the number of deep stages in the ensemble, and $b_i^{(m)}$ and $c_i^{(m)}$ are the baseline and the climate push of district i from deep stage m.

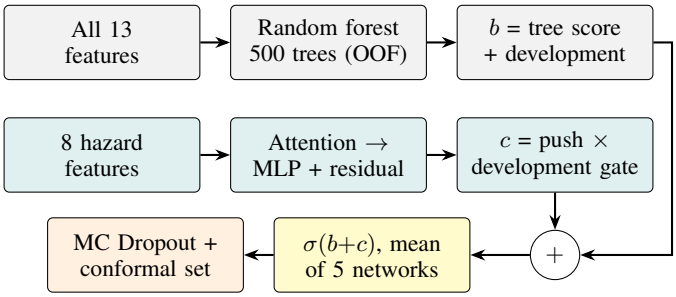


Fig. 2. B-ACPN. The deep stage starts from the random-forest score and learns a development term and a development-gated climate push.

Algorithm 1 Training B-ACPN on one fold

Require: training districts $\{(\mathbf{x}_i, y_i)\}$
standardise features with training-fold statistics
for each of 5 inner folds k **do**
  fit a random forest on the other folds; set s_i for districts in fold k
end for
refit the forest on all training districts (used for test scores)
for $m = 1$ **to** 5 **do**
  train deep stage m on $(\mathbf{x}_i, s_i, y_i)$ with early stopping
end for
centre the climate push on the training districts
calibrate the conformal threshold $\hat{q}$ on the validation split
return forest, five deep stages, $\hat{q}$

B. Training

Each network minimises the class-weighted binary cross-entropy. The training loss over the training districts is

$$\mathcal{L} = -\frac{1}{n}\sum_i \omega_{y_i}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right], \qquad (7)$$

where $\mathcal{L}$ is the training loss, n is the number of training districts, y_i is the true class of district i (1 = High, 0 = Low vulnerability), ω_{y_i} is the class weight, 0.5 divided by the share of class y_i in the training set, and $\hat{p}_i$ is the predicted probability of high vulnerability for district i.

The loss is minimised with AdamW (learning rate 3×10^{-3}, weight decay 10^{-3}, batch 64) for at most 300 epochs, stopping after 30 epochs without improvement on a stratified 15% validation split. Features are standardised on the training rows of each fold only. Each network has 780 trainable parameters, and one fold trains in 11.8 s on a CPU. Algorithm 1 summarises the procedure.

C. Uncertainty

For each district we run the deep stages 50 times with dropout active and report the standard deviation of the climate push. For classification we use split conformal prediction with nonconformity score $1-\hat{p}_y$: with $\hat{q}$ the $\lceil (n+1)(1-\alpha)\rceil/n$ empirical quantile of the calibration scores and $\alpha = 0.1$, the prediction set is $\{y : 1-\hat{p}_y(\mathbf{x}) \le \hat{q}\}$. Averaged over the folds, the sets cover the true class for 92.9% of test districts (target 90%) with an average size of 1.51.

V. Experiments and Results

A. Protocol

All six models use the same splits: repeated stratified five-fold cross-validation with five repeats (25 train/test splits, seed 42) and leave-one-state-out (LOSO) over the 21 states. The five existing models are logistic regression, a random forest (200 trees), gradient-boosted trees (100 iterations, depth 3) and a linear SVM, all trained with Spark MLlib, and a standard multilayer perceptron (13–64–32–16–1) trained with the same loop as B-ACPN. We report means over the 25 folds. Because repeated folds share training data, we compare models with the corrected resampled t-test of Nadeau and Bengio [20], which inflates the variance of the paired differences. The corrected variance of the paired differences is

$$\hat{\sigma}_c^2 = \Big(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\Big)\hat{\sigma}^2, \qquad J = 25, \qquad (8)$$

where $\hat{\sigma}^2$ is the sample variance of the 25 paired per-fold ROC-AUC differences, $\hat{\sigma}_c^2$ is the corrected variance used for the t-statistic, J is the number of train/test splits (folds), and $n_{\text{test}}/n_{\text{train}}$ is the ratio of test to training districts ($1/4$ for five-fold CV).

We use the same correction for 95% confidence intervals.

B. Comparison with existing models

Table IV gives the main comparison. B-ACPN has the highest ROC-AUC, balanced accuracy, accuracy, F1 and LOSO AUC. Its lead over gradient-boosted trees is smaller than one standard deviation and not significant (p = 0.55), so we regard the two as comparable; against the other four models the difference is significant at the 5% level. Fig. 3 shows the pooled ROC and precision–recall curves.

Table V lists further metrics. B-ACPN is best on precision, specificity, MCC, Cohen's kappa, PR-AUC, log loss, Brier score and RMSE of the predicted probability. It is not best on recall, where the linear SVM is higher, or on expected calibration error, where the random forest is lower: B-ACPN is more careful about calling a district highly vulnerable, which gives fewer false alarms at the cost of some missed districts.

C. Effect of preprocessing

To measure what preprocessing contributes, every model was also trained on the raw, unscaled features with the same folds (Table VI). B-ACPN improves on all reported metrics, for example ROC-AUC from 0.806 to 0.820. The MLP gains ROC-AUC but loses accuracy, because after scaling it labels more districts as highly vulnerable. Tree models are insensitive to feature scale, and Spark's logistic regression and linear SVM standardise internally, so their results do not change.

TABLE IV
COMPARISON WITH EXISTING MODELS (MEAN OVER 25 FOLDS)

Model	ROC-AUC	Bal. acc.	Acc.	F1	LOSO	p
B-ACPN	0.820±0.035	0.739	0.738	0.728	0.717	–
GBT	0.811±0.034	0.726	0.725	0.724	0.698	0.550
RF	0.804±0.036	0.715	0.715	0.708	0.706	0.042
MLP	0.784±0.037	0.699	0.699	0.692	0.710	0.019
SVM	0.755±0.039	0.696	0.696	0.708	0.691	0.006
LR	0.747±0.042	0.691	0.692	0.701	0.661	0.004

TABLE V
FURTHER CLASSIFICATION METRICS (↓ = LOWER IS BETTER)

Model	Prec.	Rec.	Spec.	MCC	κ	PR-AUC	LogL↓	Brier↓	ECE↓	RMSE↓
B-ACPN	0.765	0.700	0.778	0.482	0.477	0.844	0.516	0.174	0.108	0.417
GBT	0.736	0.718	0.733	0.454	0.451	0.826	0.570	0.188	0.131	0.433
RF	0.735	0.689	0.742	0.434	0.430	0.830	0.539	0.182	0.095	0.427
MLP	0.719	0.674	0.724	0.404	0.398	0.809	0.579	0.194	0.114	0.440
SVM	0.689	0.733	0.659	0.396	0.392	0.751	n/a	n/a	n/a	n/a
LR	0.688	0.718	0.665	0.386	0.383	0.746	0.611	0.206	0.101	0.454

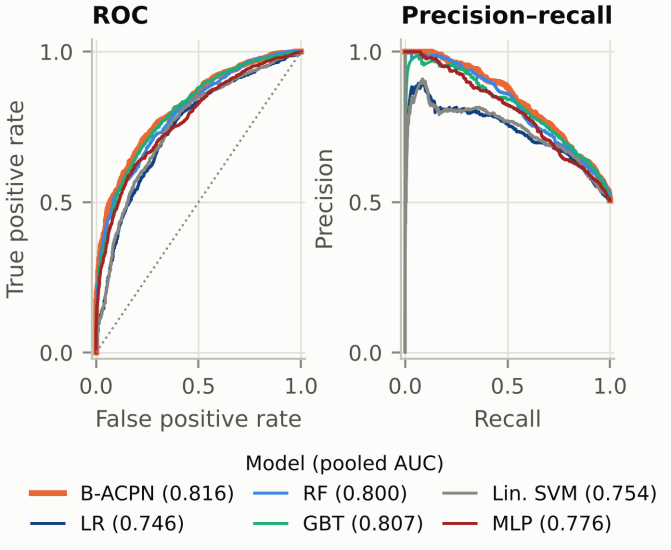


Fig. 3. ROC (left) and precision–recall (right) curves of the six models, pooled over the 25 test folds.

TABLE VI
BEFORE → AFTER PREPROCESSING (RAW VS. PREPROCESSED FEATURES)

Metric	B-ACPN	MLP	RF	LR
Accuracy	0.735→0.738	0.711→0.699	0.715→0.715	0.692→0.692
Precision	0.760→0.765	0.776→0.719	0.735→0.735	0.688→0.688
Recall	0.700→0.700	0.611→0.674	0.689→0.689	0.718→0.718
F1	0.726→0.728	0.678→0.692	0.708→0.708	0.701→0.701
Specificity	0.770→0.778	0.813→0.724	0.742→0.742	0.665→0.665
ROC-AUC	0.806→0.820	0.759→0.784	0.805→0.804	0.747→0.747
MCC	0.474→0.482	0.437→0.404	0.434→0.434	0.386→0.386
Log loss↓	0.528→0.516	0.588→0.579	0.539→0.539	0.611→0.611
RMSE↓	0.422→0.417	0.445→0.440	0.427→0.427	0.454→0.454

D. Ablation

Table VII removes one part of B-ACPN at a time. Removing the tree stage or the ensemble costs about 0.02 ROC-AUC, the largest drops. Removing the climate push, the attention or the gate changes accuracy by less than one standard deviation; these parts are kept because they provide the per-district explanation, not because they add accuracy.

E. Hyperparameter tuning

We tuned the deep stage over 18 settings (hidden units 8, 16, 32; dropout 0.1, 0.3, 0.5; learning rate 0.001, 0.003) on the same 25 folds with the tree stage fixed, choosing on CV only and checking on LOSO. All settings lie between 0.817 and 0.820 ROC-AUC; the best (16 units, dropout 0.3, learning rate 0.001) improves on the default by 0.001 in CV and 0.002 in LOSO, far below the fold SD of about 0.035, so B-ACPN is insensitive to these settings and we keep the default.

TABLE VII
ABLATION OF B-ACPN (ROC-AUC, 25 FOLDS)

Variant	ROC-AUC	Change	p
Full B-ACPN	0.820	–	–
Deep stage only (no tree stage)	0.800	-0.019	0.50
Tree stage only (no deep stage)	0.818	-0.002	0.95
Without climate push	0.816	-0.004	0.88
Without development gate	0.821	+0.001	0.96
Without feature attention	0.817	-0.003	0.93
Single network (no ensemble)	0.801	-0.018	0.58

TABLE VIII
MEAN CLIMATE PUSH BY GROUP

Group	n	Push	Push > 0
Developed, low vul. (expected)	143	-0.10	45%
Less developed, high vul. (expected)	145	+0.13	86%
Anomaly A: developed, vulnerable	92	+0.09	82%
Anomaly B: less developed, resilient	89	-0.17	46%

F. What the climate push explains

Fig. 4 and Table VIII show the out-of-fold climate push per group. Developed but vulnerable districts (anomaly A) have a positive mean push of +0.09, against -0.10 for developed districts that are not vulnerable (p = 1.6×10^{-5}). Less developed but resilient districts (anomaly B) have a negative push of -0.17, against +0.13 for the expected group (p = 4.5×10^{-7}). The raw data agree: anomaly A districts have about twice as many tropical nights as expected developed districts (91 vs 45 per year, p = 0.001). Attention gives heavy-rain days the largest mean weight. The Spark climate regimes, found without the model, also differ in push (+0.05 for the larger regime and -0.24 for the smaller one), which supports that the push follows real climate patterns. At the regression level, adding the eight hazards to development raises the explained variance of the index from 0.127 to 0.334.

G. Comparison with the CEEW ranking

As an independent check we matched the 463 districts of the CEEW 2021 hotspot table [1] to Census codes by name

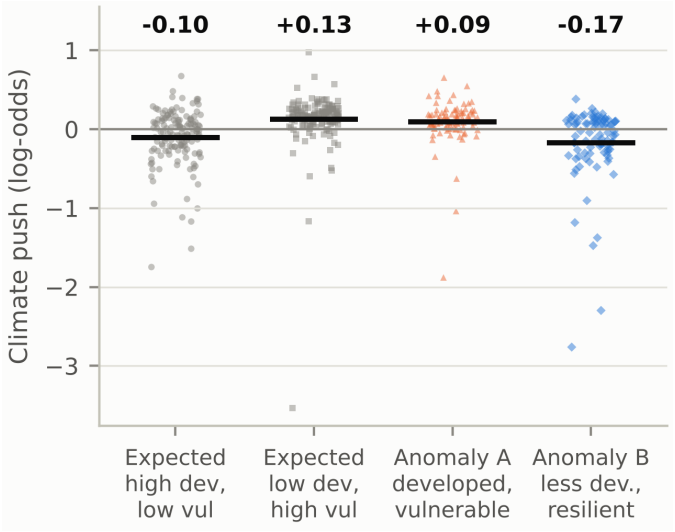


Fig. 4. Out-of-fold climate push of every district by group; black bars and the numbers above mark group means.

(ambiguous names dropped); 350 of our 469 districts appear in both. The two indices barely agree: Spearman $\rho = 0.02$ ($p = 0.66$) between the CEEW index and our NDAP target, 0.04 with B-ACPN's predicted risk, and 0.10 ($p = 0.05$) with the climate push; high/low classes agree for 51% of districts ($\kappa = 0.02$). CEEW's index is driven by hazard exposure, while the NDAP indicators are mainly socio-economic, so which districts count as vulnerable depends on the index; our conclusions hold for the NDAP index only.

H. Interactive dashboard

To make the explanations usable outside the code, we built a browser dashboard (Fig. 5) that reads the out-of-fold results directly. A user searches for any of the 469 districts and sees its group, baseline, climate push with its Monte Carlo Dropout uncertainty, the dominant climate factor and the attention weights of all eight hazards, next to the most climate-driven districts and the mean push of each group. The page is a single static HTML file generated from the results, so it runs offline and always shows the same numbers as the tables.

I. Pipeline scaling

On the single node, MapReduce took 55, 140 and 333 s for 1, 10 and 30 years of input (two runs each). Runtime grows almost linearly with input, while throughput rises from about 4,300 to 21,000 records per second as the fixed job start-up cost is shared over more data.

VI. Discussion

Three lessons stand out. First, on 469 districts a tree ensemble is a hard baseline, and the network was most useful as a correction on top of it rather than as a replacement; design choices were compared with an inner cross-validation inside the training folds, so the test folds were not used to pick the model. Second, the explanation is not free accuracy: the

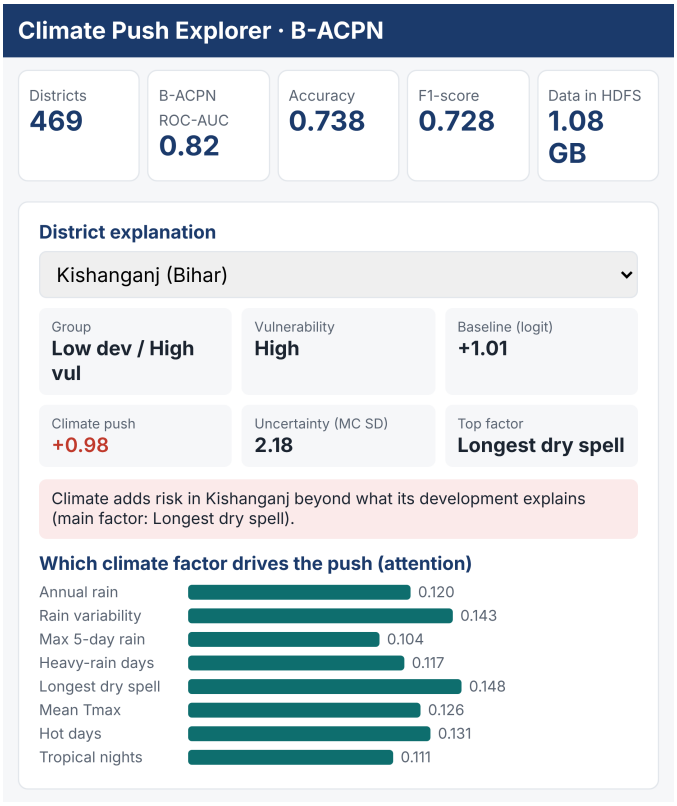


Fig. 5. Climate Push Explorer dashboard built on the B-ACPN results (district view), here showing Kishanganj (Bihar), the most climate-driven district.

climate push, attention and gate are kept for interpretability, and the ablation shows that honestly. Third, the Big Data pipeline earns its place through repeatability and checks rather than speed on one machine: each job has an independent pandas check, and the same scripts run on a real cluster.

The work has clear limits. Development comes from Census 2011 while the index is from 2021. Temperature is on a coarse 1° grid. Only five development variables survive without imputation, and 469 districts in 21 states give fold-to-fold standard deviations of about 0.035 ROC-AUC. Hadoop ran on a single node, so no multi-machine speed-up is claimed. The advantage of B-ACPN over gradient-boosted trees is not statistically significant.

VII. Conclusion and Future Work

We combined a Hadoop pipeline over 30 years of daily IMD climate data with a hybrid model, B-ACPN, to explain why development does not always predict climate resilience in Indian districts. B-ACPN matches or exceeds five existing models on every main metric and, unlike them, assigns each district a climate push with uncertainty, which shows that climate explains much of why developed districts remain vulnerable. Future work will extend the climate record to 1991–2024, model the CEEW index as a second target, and move the pipeline to a multi-node cluster.

REFERENCES

[1] A. Mohanty and S. Wadhawan, "Mapping India's climate vulnerability: A district-level assessment," Council on Energy, Environment and Water (CEEW), New Delhi, 2021.

[2] Department of Science and Technology, Government of India, "Climate vulnerability assessment for adaptation planning in India using a common framework," New Delhi, 2020.

[3] IPCC, *Climate Change 2022: Impacts, Adaptation and Vulnerability*. Contribution of Working Group II to the Sixth Assessment Report. Cambridge, U.K.: Cambridge Univ. Press, 2022.

[4] D. S. Pai, L. Sridhar, M. Rajeevan, O. P. Sreejith, N. S. Satbhai, and B. Mukhopadhyay, "Development of a new high spatial resolution ($0.25° \times 0.25°$) long period (1901–2010) daily gridded rainfall data set over India and its comparison with existing data sets over the region," *MAUSAM*, vol. 65, no. 1, pp. 1–18, 2014.

[5] A. K. Srivastava, M. Rajeevan, and S. R. Kshirsagar, "Development of a high resolution daily gridded temperature data set (1969–2005) for the Indian region," *Atmospheric Science Letters*, vol. 10, no. 4, pp. 249–254, 2009.

[6] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[7] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[8] V. K. Vavilapalli *et al.*, "Apache Hadoop YARN: Yet another resource negotiator," in *Proc. 4th ACM Symp. Cloud Computing (SoCC)*, 2013, pp. 1–16.

[9] A. Thusoo *et al.*, "Hive: A warehousing solution over a map-reduce framework," *Proc. VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, 2009.

[10] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[11] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2016.

[12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[13] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[14] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, vol. 35, 2022.

[15] S. Ö. Arık and T. Pfister, "TabNet: Attentive interpretable tabular learning," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 8, 2021, pp. 6679–6687.

[16] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[18] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, pp. 1050–1059.

[19] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv:2107.07511, 2021.

[20] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[21] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.